

An Improved Particle Swarm Optimization Algorithm Based On Entropy And Fitness Of Particles

Huafen Yang*, Yong Li, Lihua Yang, Qian Wu

Department of Computer Science and Engineering, Qujing
Normal College, Qujing 655000, China
corresponding author's email: sunnyyh@126.com

ABSTRACT—Particle swarm optimization (PSO) is an algorithmic technique for optimization by solving a wide range of optimization problem. This paper presents an improved PSO. The proposed algorithm consists of two parts. Firstly, population initialization method based on entropy is proposed. Secondly, an improved accelerated learning coefficient is proposed. In this algorithm, a modified velocity updating formula of the particle is used, where the randomness in the course of updating particle velocity and the acceleration coefficient is relatively decreased. The entropy of each dimension is calculated to decrease the randomness of swarm and increase population diversity. Each particle has a different learning rate according to its fitness during evolution, which can balance the global and local searching ability of the population and avoid falling into local optimum. Experimental results show that, the proposed algorithm remarkably improves the ability of PSO to jump out of the local optima and significantly enhance the convergence precision.

Keywords- Particle swarm optimization; Initialization population based on entropy; Acceleration coefficient

I. INTRODUCTION

Particle swarm optimization (PSO) is a population-based stochastic optimization technique developed by Dr Eberhart and Dr Kennedy in 1995, inspired by the social behaviour of birds[1-2]. PSO algorithm has been successfully applied to a wide range of application in many domains. PSO algorithm has been successfully applied to many optimization problems[3-5]. However, PSO does exhibits some disadvantages: it is easy to be trapped in local optima, and the convergence rate decreased in the later period of evolution and the accuracy the algorithm can achieve is limited. The initial population is randomly generated and has a certain blindness, which may cause the result to deviate from the optimal solution. scholars have proposed many improved algorithms to enhance the search ability and avoid convergence to local minimum[6]. Edmarlyn M. Porras, Arnel C. Fajardo and Ruji P. Medina proposed an improved PSO using constriction coefficient method and genetic operators were integrated to explore the search space[7]. Oussama Aoun, Malek Sarhani, and Abdellatif El Afia. Proposed an automatic control of acceleration coefficients [8]. Kuang Fangjun, Jin Zhong and Xu Weihong use Tent chaotic opposition-based learning to improve initialization population, and avoid premature convergence[9]. XU Xiaobo, ZHENG Kangfeng, LI an,

et al. made the motion of particles with characteristics of chaos, and proposed a new chaos-particle swarm optimization model[10]. Chaos optimization algorithm (COA) is used to to optimize particle swarm optimization algorithms. The chaos optimization method has global asymptotic convergence, and is easy to jump out of local minima[11]. In this paper, an adaptive chaos optimization particle swarm optimization algorithm is proposed to improve the global convergence of the PSO algorithm, which can improve the local search ability and improve the convergence speed.

This paper proposes a particle swarm optimization algorithm based on entropy and individual fitness. In order to control the distribution of the initial particles in the solution space, the entropy of each dimension is calculated when initializing particles. Initial population is based on entropy, which can decrease the randomness of swarm and increase population diversity. In this algorithm, a modified velocity updating formula of the particle is used, where the randomness in the course of updating particle velocity and acceleration coefficient is relatively decreased. Each particle has a different learning rate according to its fitness during evolution, which can balance the global and local searching ability of the population and avoid falling into local optimum. Experimental results show that, the proposed algorithm remarkably improves the ability of PSO to jump out of the local optima and significantly enhance the convergence precision.

II. IMPROVED PARTICLE SWARM OPTIMIZATION

Although particle swarm optimization algorithm has been widely used in various optimization problems, and scholars have proposed various improvements. In order to improve the search efficiency and speed up the convergence, the population initialization method based on entropy and the acceleration coefficient based on fitness ranking were proposed.

A. Method of Initializing Particles

The initial position of the particles determines the diversity of the particle swarm. If the population is concentrated and distributed in a local area, the diversity of the population is poor, which will lead to the search time is too long to find the optimal value. In order to make the initial particle position in the solution space as evenly as possible, calculate the entropy of each dimension in the initial particle position, so as to control the distribution of the initial particle. If

there are $N-1$ particles in the population that meet the requirements, the entropy of the i th dimension of the N th particle can be expressed as follows:

$$S_i = \sum_{m=1}^{N-1} (-\ln G_{mN}^i) \quad (1)$$

$$G_{mN}^i = 1 - \frac{|y_m^i - y_N^i|}{|C_{\max}^i - C_{\min}^i|} \quad (2)$$

Where $C_{\max}^i$, $C_{\min}^i$ respectively represent the upper and lower bounds of the component in the individual. N is the number of particles in the population that meet the requirements. $N \leq M$, M is population size. G_{mN}^i is the probability that the i th particle and the m th particle have the same component. y_m^i is the value of d -dimension of the i th particle. y_N^i is the value of d -dimension of the N th particle. It can be seen from equation (2) that the closer the i th dimension of two particles, the bigger the value of G_{mN}^i . When the i th dimension is very close, $G_{mN}^i \rightarrow 1$, $\lim_{G_{mN}^i \rightarrow 1} (-\ln G_{mN}^i) = 0$; whereas $G_{mN}^i \rightarrow 0$, $\ln G_{mN}^i \rightarrow -\infty$, $\lim_{G_{mN}^i \rightarrow 0} (-\ln G_{mN}^i) = \infty$. The entropy of a population can be defined as the average of the i th dimensional entropy of all individuals in the population.

$$S = \frac{1}{N} \sum_{j=1}^N |S_j| \quad (3)$$

The algorithm for the initial particle position is as follows:

Step1: Determine critical entropy S_0 ;

Step2: According to formula (2), the same probability of the i th dimension of particles m and n is calculated. According to equation (1), the entropy S_i of the i th dimension of the N th particle is determined. If $S_i > S_0$, the component is accepted, otherwise it is regenerated;

Step3: Repeat step2 until the number of particles generated reaches the population size.

B. Improvement of Acceleration Coefficient

The acceleration coefficient is mainly used to balance the exploration and exploitation of particle swarm optimization. From the position update formula, it can be seen that the larger the acceleration coefficient is, the faster the particle learns from other particles, and the smaller the acceleration coefficient is, the slower the particle learns from other particles. The worse a particle is, the more it should learn from other particles. The worst particle has a higher speed, which allows it to acquire the global search capability. On the contrary, assigning a smaller speed to the best particle allows it to search around itself.

At the end of each search, the fitness of the particles was sorted. The fitness of the best particle found by the i th particle up to now is P_i , and the maximum fitness and minimum fitness of the population are $P_{\max}$ and $P_{\min}$ respectively. The learning speed factor can be

expressed as:

$$F_i = \frac{fit_i - fit_{\min}}{fit_{\max} - fit_{\min}} \quad (4)$$

Where $fit_{\min} = \min(fit_1, \dots, fit_N)$, $fit_{\max} = \max(fit_1, \dots, fit_N)$, $i = 1, 2, \dots, N$. fit_i is the fitness of the i th particle that has been found so far. $fit_{\min}$ is the global minimum fitness. $fit_{\max}$ is the global maximum fitness.

The acceleration coefficient based on fitness can be expressed as:

$$c_i = (c_2 - c_1) * F_i + c_1 \quad (5)$$

Where c_1 is the lower bound and c_2 is the upper bound of the acceleration coefficient, $c_1 = 1.25$, $c_2 = 1.75$, $i = 1, 2, \dots, N$.

The acceleration coefficient of the population is in the range of $[c_1, c_2]$. Through this linear transformation, each particle has a different learning rate during evolution. The worst particle has better global search ability, and the best particle has better local search ability. This can balance the global and local searching ability of the population. The acceleration coefficient of each particle is changed according to its fitness value, not according to iterations. In this method, c_1 and c_2 vary monotonically with F_i , which can be well adapted to the search environment. If a better global optimal solution is found, all particles in the population will have a higher learning speed. At the same time, the best particle will wait quietly for other particles to fly towards it. If the population requires a deep search, all particles have a small learning rate, which leads to a local search of the population.

C. The Proposed Strategy

Updates to speed and position can be expressed as:

$$v_i^{t+1} = wv_i^t + c_1 r_i^t (pbest_i^t - x_i^t) + c_2 r_i^t (gbest_i^t - x_i^t) \quad (6)$$

$$x_{i+1}^t = x_i^t + v_i^{t+1} \quad (7)$$

Many common improved algorithms use an acceleration factor for the same generation of particles. The fitness of each particle is not the same, their need for learning speed is not the same. This paper proposes a learning speed based on individual fitness. Each particle has a different learning speed in each generation.

III. EXPERIMENTATIONS

Five benchmark functions are listed in Table 1. All the functions were tested on 30 dimensions. The population size of all the PSOs was set 100.

Rosenbrock function:

$$f_1(x) = \sum_{i=1}^{n-1} (100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2)$$

Generalized Rastrigrin function:

$$f_2(x) = \sum_{i=1}^{n-1} (x_i^2 - 10 \cos(2\pi x_i) + 10)$$

Generalized Griewank function:

$$f_3(x) = \frac{1}{4000} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$$

Schaffers f6 functions:

$$f_4(x) = 0.5 + \frac{(\sin \sqrt{(x_1^2 + x_2^2)})^2}{(1 + 0.001(x_1^2 + x_2^2))^2}$$

Sphere function :

$$f_5(x) = \sum_{i=1}^n x_i^2$$

Table 1 Benchmark configurations

Funtion	Name	Trait	Domain	Minimum
f_1	Rosenbrock	unimodal	$[-30,30]$	0
f_2	Rastrigrin	Multimoda 1	$[-5.12,5.12]$	0
f_3	Griewank	Multimoda 1	$[-600,600]$	0
f_4	Schaffers f6	Multimoda 1	$[-100,100]$	0
f_5	Sphere	unimodal	$[-100,100]$	0

When optimizing the function f_1 with the traditional particle swarm optimization algorithms, the worst fitness is $11.60986905035565 \times 10^5$, and the best fitness value is $0.003573324560956 \times 10^5$. The worst value obtained by the entropy-based particle swarm optimization algorithm is $0.35684523975569499 \times 10^5$, and the best value is 0. The worst fitness value obtained by IPSO algorithm is $0.2526924664836431 \times 10^5$. The optimal value is 0, the optimal value is obtained after only two generations of evolution. The fitness change is shown in figure 1.

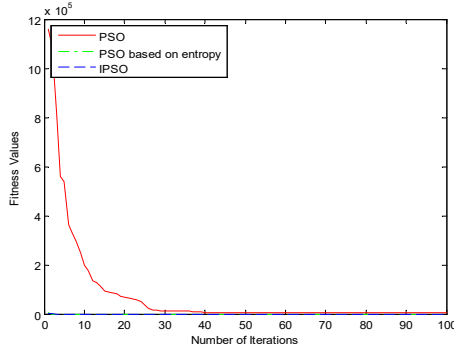


Figure 1 The fitness variation image of function f_1

When optimizing the function f_2 with the traditional particle swarm optimization algorithms, the worst fitness is $10.68786534627818 \times 10^4$, and the best fitness value is $0.0491002172381483 \times 10^4$. The worst value obtained by the entropy-based particle swarm optimization algorithm is 1.248576623168161 , and the best value is 0. The worst fitness value obtained by IPSO algorithm is 2.202546579480672 . The optimal value is 0, the optimal value is obtained after only two generations of evolution. The fitness change is shown in figure 2.

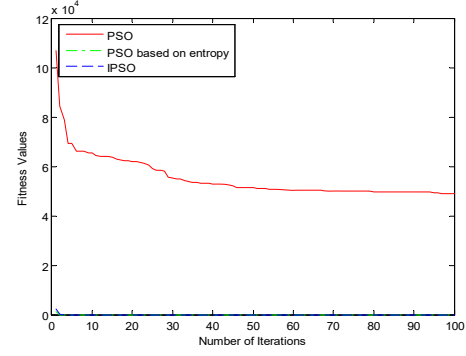


Figure 2 The fitness variation image of function f_2

When optimizing the function f_3 with the traditional particle swarm optimization algorithms, the worst fitness is $6.110148021723833 \times 10^5$, and the best fitness value is $0.491382171480023 \times 10^4$. The worst value obtained by the entropy-based particle swarm optimization algorithm is $0.202654806725479 \times 10^5$, and the best value is 0. The worst fitness value obtained by IPSO algorithm is 3.480654672202579 , and the best value is 0, the optimal value is obtained after only two generations of evolution. The fitness change is shown in figure 3.

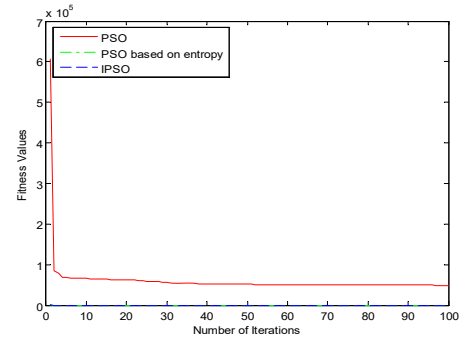


Figure 3 The fitness variation image of function f_3

When optimizing the function f_4 with the traditional particle swarm optimization algorithms, the worst fitness value was $2.068786534627818 \times 10^5$, and the best fitness value was $0.491081021723483 \times 10^4$. The worst value obtained by the entropy-based particle swarm optimization algorithm is $1.236121857668142 \times 10^4$, and the best value is 0. The worst fitness value obtained by IPSO algorithm is 2.01254854602567279 , and the optimal value is 0, the optimal value is obtained after only two generations of evolution. The fitness change is shown in figure 4.

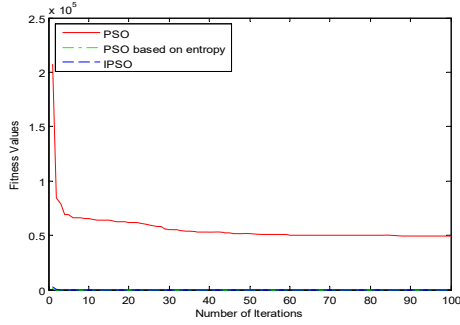


Figure 4 The fitness variation image of function f_4

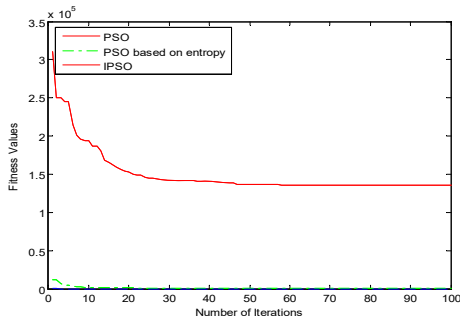


Figure 5 The fitness variation image of function f_5

When optimizing the function f_5 with the traditional particle swarm optimization algorithms, the worst fitness was $3.254784034280686 \times 10^5$, and the best fitness value from evolution 100 was $1.353023665113391 \times 10^5$. The worst fitness is 1.683443964247381 when particle swarm optimization based on entropy is adopted, and the best fitness value obtained after 20 generations of evolution is 0.41504405756288 . When IPSO algorithm is used for optimization, the worst fitness is 2.067265720254948 , the best fitness value is 0, and the optimal value is reached after only two generations of optimization.

LDW, TVAC and APSO[12] can all find the optimal value, but evolutionary algebra is more than IMPSO algorithm. It can be seen from the above results that the global optimal value cannot be found when using the traditional particle swarm optimization algorithm to optimize the function. Particles are randomly initialized, which may be concentrated in a certain region, and the diversity of the population is poor, which will lead to the algorithm search time is too long to find the optimal value. In order to make the position of the initial particles as evenly distributed as possible in the solution space, the entropy based particle swarm optimization algorithm is adopted to calculate the entropy of each dimension when the position of the initial particles is located, so as to control the distribution of the initial particles in the solution space. Therefore, the worst fitness value found is close to the optimal value, and the global optimal value can be found after a little evolutionary algebra. IPSO algorithm adopts the entropy-based particle

initialization method and the acceleration coefficient based on individual fitness. In the process of evolution, each particle has its own acceleration coefficient. The worse the particle is, the stronger the global search ability is, and the better the particle is, the stronger the local search ability is. This method can better balance the global and local search ability of the population. The acceleration coefficient of each particle is changed according to its fitness value, not according to the number of iterations.

IV. CONCLUSIONS

The algorithm proposed in this paper is easy to implement. The experiment of this algorithm gives an obvious result: the ability to jump out of local optima has been significantly improved, the convergence accuracy and speed are improved obviously. Compared with the traditional PSO and other PSO, the improved PSO has more advantages in finding the global optimal solution and the quality of the solution.

ACKNOWLEDGEMENTS

This work is supported by research fund project of yunnan education department (Grant Nos. 2019J0602)

REFERENCES:

- [1] J. Kennedy, R.C. Eberhart, "Particle swarm optimization." Proceedings of the IEEE International Conference on Neural Networks, 1995, vol.4, IEEE press, pp.1942–1948.
- [2] R. Eberhart, J. Kennedy. "A new optimizer using particle swarm theory." Proceedings of the Sixth International Symposium on Micro Machine and Human Science, pp.39–43.
- [3] Zhi-Hui Zhan, Jun Zhang, Yun Li, and H.S.-H. Chung. "Adaptive Particle Swarm Optimization." IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, 2009, vol. 39, no.6, pp.1362–1381.
- [4] Edmarlyn M.Porrás, Arnel C.Fajardo, Ruji P.Medina. "Solving Dietary Planning Problem using Particle Swarm Optimization with Genetic Operators." Proceedings of the 3rd International Conference on Machine Learning and Soft Computing, 2019, pp. 55-59.
- [5] Oussama Aoun, Malek Sarhani, and Abdellatif El Afia. "Hidden markov model classifier for the adaptive particle swarm optimization." In Recent Developments in Metaheuristics, 2018, pp. 1–15.
- [6] KUANG Fang-jun, XU Wei-hong, JIN Zhong. "Hybridization algorithm of Tent chaos artificial bee colony and particle swarm optimization." Control and Decision, 2015, pp. 839-847.
- [7] XU Xiaobo, ZHENG Kangfeng, LI Dan, et al. "New Chaos-Particle Swarm Optimization Algorithm." Journal on Communications, 2012, pp.24-31.
- [8] REN Jin-xia, YANG Shuai. "Chaos Coded Particle Swarm Optimization Algorithm and Its Application." COMPUTER SIMULATION, 2013, pp. 299-302.
- [9] Xuemin Yang, Jinsha Yuan, Jiangye Yuan, Huina Mao. "A modified particle swarm optimizer with dynamic adaption." Applied Mathematics and Computing, 2007, vol.189, pp.1205-1213.
- [10] Ricardo Brito, Simon Fong, Yan Zhuang, Yaoyang Wu. Generating Neural Networks with Optimal Features through Particle Swarm Optimization. Source: ACM International Conference Proceeding Series, p 96-101, December 20,

2017, BDIOT 2017 - Proceedings of the International Conference on Big Data and Internet of Thing.

- [11] Porras, Edmarlyn M. (Department of Computer Studies, Technological Institute of the Philippines, Quezon City, Philippines); Fajardo, Arnel C.; Medina, Ruji P. Solving Dietary Planning Problem using Particle Swarm Optimization with Genetic Operators. ACM International Conference Proceeding Series, p 55-59, January 25, 2019, ICMLSC 2019 - Proceedings of the 3rd International Conference on Machine Learning and Soft Computing.
- [12] Riofrio, Daniel (Escuela Politécnica Nacional, Ladrón de Guevera E11-253, Quito; 170517, Ecuador); Luan, Shuang; Zhou, Jun; Ma, Lijun. Particle swarm optimization for radiation therapy planning .Source: BCB 2015 - 6th ACM Conference on Bioinformatics, Computational Biology, and Health Informatics, p 250-257, September 9, 2015, BCB 2015 - 6th ACM Conference on Bioinformatics, Computational Biology, and Health Informatics